

Automated machine learning approach for time series classification pipelines using evolutionary optimization

Ilia Revin^{*}, Vadim A. Potemkin, Nikita R. Balabanov, Nikolay O. Nikitin

ITMO University, Saint Petersburg, Russia

ARTICLE INFO

Article history:

Received 25 August 2022

Received in revised form 17 February 2023

Accepted 14 March 2023

Available online 21 March 2023

Keywords:

Time series classification

AutoML

Machine learning

Composite AI

ABSTRACT

Automated machine learning has the ability to improve the efficiency of time series classification due to the ability to combine multiple feature extraction methods and classification models. In the paper, we propose a flexible AutoML approach that combines multiple feature generation strategies (spectral, wavelet, topological, quantile) and classifiers as parts of the modeling pipeline. It allows obtaining a more robust and lightweight solution for the time series classification problem. Generation of the pipeline is based on an evolutionary algorithm.

Comparison with approaches with the highest results was conducted on the UEA/UCR archive for modeling quality analysis. The proposed approach allows, on the one hand, to solve the task of time series classification automatically and could be used as part of an industrial data processing pipeline. On the other hand, it could be used as a tool for modeling and studying natural process properties described as time series.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

The classification task for univariate and multivariate time series arises in many fields of science and technology [1,2]. Speech recognition [3] and analysis of seismological problems [4], meteorological problems [5], medical problems [6] and industrial problems [7] can be taken as an example.

The most direct pipeline for time series classification (TSC) has the following steps: (1) splitting the available time series dataset into training and test subsets; (2) choosing and applying a feature extraction strategy; (3) fitting a classification model using feature vectors obtained from the initial TS; (4) making predictions on the test subset.

At the moment, there are several issues for which existing solutions are either not developed or can be improved:

1. Automation of the design of the feature space. Some of the existing libraries [8,9] for feature extraction from time series often implement a limited set of base methods that do not take into account all of the time series properties (for example, shape, trend, cyclic components analysis). Usually, feature engineering aims to make the feature space dimensionality as small as possible to increase the stability

of classification results. However, the optimal subset of features differs for every specific dataset. Also, these features can belong to different domains (time-based domain or frequency-based domain);

2. Classification model selection strategy. Existing solutions to select a time series classification model implement only two possible strategies. The first one is the so-called “black-box” approach, which is the use of deep learning architecture on initial data [10,11]. The second is the expert model selection and hyperparameters tuning (in automatic or manual mode) on extracted time series features. However, generalizing the selected models and hyperparameters to new data is an extremely difficult and often unsolvable task;
3. AutoML for time series classification problem. Automatic selection of an optimal classification model for time series does not have a ready-to-use solution. Most state-of-the-art AutoML solutions (such as Autogluon, H2O, TPOT, and sklearn) are designed to work with tabular data. Existing automated time series classification solutions (mcfly, tslearn, pyts, etc.) are affected by the problems described above. This way, automating the creation of a machine learning pipeline for time series classification problems, which consist of feature generators and classification models, is an unsolved and relevant problem.

In the paper, we propose the data-driven automated time series classification approach FEDOT-TSC, which intends to solve

^{*} Corresponding author.

E-mail addresses: revin@itmo.ru (I. Revin), vadim_potemkin@itmo.ru (V.A. Potemkin), nrbalabanov@itmo.ru (N.R. Balabanov), nnikitin@itmo.ru (N.O. Nikitin).

problems described above. The main idea is to combine an automated selection of features in one feature space and an automated classification model design using a graph-based pipeline representation and evolutionary optimization. We choose this approach for several reasons. Firstly, optimization methods based on evolutionary algorithms have shown their effectiveness in solving the problem of time series classification [12–14]. Also a evolutionary optimization algorithm has shown its effectiveness in solving more generalized problems [15,16]. At the same time, there are a number of alternative optimization methods for solving TSC problem [17,18]. For automatic feature generation, we implemented several methods based on the time series spectral, statistical and geometrical properties.

For the machine learning pipeline creation, represented as an acyclic graph, we used an approach based on an evolutionary algorithm derived from [16]. This pipeline combines multiple methods of feature extraction and machine learning models. The aim of the evolutionary optimizer is to obtain the effective but computationally lightweight pipeline avoiding over-complicated solutions.

Since the proposed approach is fully automated, it could be used by specialists without machine learning or data science experience for application tasks in their professional domain, where the primary data type is time series. The proposed approach is available as part of the novel open source automated machine learning framework [19].

The paper is organized as follows. Section 2 describes the existing approaches for time series classification and state-of-the-art solutions summary. Section 3 contains the research problem statement and two main hypotheses that are disclosed further. Section 4 comprehensively describes the proposed approach to time series classification. Section 5 is designed to give the reader an in-depth understanding of the conducted research process, and the results are discussed in Section 6.

2. Related works

It has to be mentioned that deep learning methods have gained strength in recent years, but those models are often considered “black boxes” due to the lack of interpretability [20]. There are also approaches such as [21] and [11], which allow obtaining superior or equal to state-of-the-art solutions results. Those solutions are based on a convolutional neural network, so comparing them is incorrect.

In this regard, we aim to overview the various approaches that can be used to classify the time series: from state-of-the-art algorithms to the AutoML tool. Special attention is given to the automated methods as most comparable with the proposed approach.

2.1. Time series classification algorithms

In the study, [22] time series classification algorithms are divided into the following five categories:

- Distance-based (e.g. KNN with dynamic time warping);
- Interval-based (e.g. TimeSeriesForest);
- Dictionary-based (e.g. BOSS, cBOSS)
- Frequency-based (e.g. RISE);
- Shapelet-based (e.g. Shapelet Transform Classifier, Fast Shapelet Tree).

We analyzed existing algorithms from each group to find out the model idea and its pros and cons. Moreover, in this section, existing solutions for time series classification are considered.

DTW Features (DTW_F)

Dynamic Time Warping (DTW), in combination with 1-nearest neighbor (1-NN), has been challenging to outperform on the time series classification task for decades [23].

Classic DTW is intended to calculate the path's cost between two-time series - DTW distance [24].

DTW-Features (DTW-F) considers DTW distances as feature vectors further used in the SVM algorithm.

The Shapelet Transform Classifier

This algorithm is based on a data mining primitive called time series shapelets, which are representative of the whole classes [25,26].

The following advantages of shapelets-based time series classification methods could be distinguished:

1. For some time series, shapelets can be significantly more accurate and efficient because they can “spot” their characteristic local features;
2. Compared to many other classification methods, shapelets can provide significantly higher classification speed.

Random Interval Spectral Ensemble (RISE) exploits the concept of tree-based ensembles (random forest or time series forest) [27]. RISE extracts spectral features from each random interval, which makes it possible to improve classification accuracy, particularly for long series where spectral features can change over time [28].

TS-CHIEF stands for Time Series Combination of Heterogeneous and Integrated Embeddings Forest, an ensemble algorithm that complements Proximity Forest's splitters with dictionary-based and interval-based splitters [29]. As mentioned in [29], the algorithmic contributions are three-fold: it uses a dictionary-based method BOSS, and an interval-based method RISE to develop two appropriate tree-splitters, which are then integrated with those introduced by Proximity Forest. It means that all of these types could be used in the tree nodes.

InceptionTime is proposed in the article [10] that describes the ensemble of convolutional neural networks, which use a different size of the convolution window to generate a feature space that best represents the original time series.

WEASEL method is proposed in [1] and stands for Word ExtrAction for time Series cLAssification. The distinct feature of the method is a transformation of time series into feature vectors, using a sliding-window approach, which is further analyzed through a machine learning classifier.

Proximity Forest is an effective method in terms of time consumption [30]. The models in his approach are ensembles of highly randomized Proximity Trees. The main feature of it is that Proximity Trees branch on the proximity of time series to one exemplar time series or another.

OS-CNN stands for Omni-Scale block (OS-block) for 1D-CNNs, where kernel sizes are decided by a simple and universal rule [31]. It is one of the latest deep learning methods at the moment for time series classification that also achieves state-of-the-art performance.

2.2. Automated solutions for time series classification

The last decade has brought to life numerous projects related to time series analysis. Most are based on the scikit-learn library or PyTorch/Keras for ML and Deep Learning. Table 1 combines and briefly describes existing solutions related to the time series classification problem, including mcfly [32], tsfresh [9], pyts [33], sktime [34], tslearn [35], and FEDOT [15,16]. This information led to the following conclusion: even though there are several solutions for the task in the AutoML domain, none of them implement the solution for the problems mentioned before.

Table 1
Summary of existing solutions for automated time series classification.

Solution	Type	Description	Pros	Cons
mcfly	AutoML tool	Generates models of 4 different architecture types (CNN, DeepConvLSTM, ResNet, InceptionTime) and exploits random search among them	(a) Visualization of models performance (b) Specifically designed for time series classification problem (c) Automated hyperparameters selection (d) Able to work with raw data	(a) Less interpretability of the model (b) Limited set of models and the use of deep learning approach may be inefficient for certain time series
tsfresh	Python package	Automatically calculates a large number of time series characteristics	(a) Custom feature creation option (b) Evaluation of explaining power (c) Feature filtering	(a) Only operate with statistical and spectral features, but lack of window features such as trajectory matrix (b) Cannot operate with unevenly sampled data and categorical data
pyts	Python package	Open-source Python package for time series classification. Implements algorithms, preprocessing and data set loading utilities	(a) Build on top of well-known libraries, such as scikit-learn, numpy, scipy (b) Utilizes Numba instead of Cython for optimization	(a) Classification and transformation are done with several pre-defined algorithms (b) Does not support dimensionality reduction as preprocessing operation
sktime	Python library	A library for time series analysis in Python	(a) Community-driven project (b) Include a huge variety of distance-based, kernel-based, feature-based and interval-based methods	(a) Allow only manually select classification algorithm (b) Feature extraction is based on tsfresh package or canonical time-series characteristics
tslearn	Python package	Machine learning toolkit for time series analysis	(a) Besides classification supports clusterization and regression (b) Include an API reference (c) Implements non-trivial metrics	(a) Contains of limited set of models for classification and preprocessing (b) Does not have an option of automated selection of classification algorithm
FEDOT TSC	AutoML framework	Python framework based on FEDOT AutoML, designed for time series classification	(a) Exploits series of light-weight models instead of heavy-weight deep learning algorithms	(a) Long convergence due to the evolutionary algorithm

2.3. Remaining problems

In existing approaches, feature extraction has been sufficiently studied and developed using frequency-based and time-based methods (with and without interval-based or window-based estimates). However, methods for ensembling features among themselves and methods for extracting features that characterize specific properties of time series are not widely used among the presented approaches.

Also, the disadvantages of the existing approaches include a relatively straightforward and non-flexible design of classification models, either simple linear classifiers (KNN, SVM) or complex deep learning models.

In addition, the high computational complexity of end-to-end approaches for time series can be considered the weak side. For example, training the HIVE-COTE model on a dataset containing 1500 time series can take eight days, and the training time of the TS-CHIEF model on 130 000-time series can reach two days [29].

Based on the review of existing solutions, we conclude that there are solutions that implement either model selection or feature space generation. However, there are no solutions that would combine these two tasks. In our opinion, such a tool should be functionally flexible and lightweight, in contrast to deep learning approaches.

3. Theory

The classification of time series is the task of constructing a classifier based on input data presented as a time-ordered data sequence. The mathematical formulation of the problem can be presented in the following form:

$$F : X \rightarrow \mathbb{D} \quad (1)$$

where:

X is a time series of length $t = [x_1, x_2, \dots, x_t]$, $x_i \in R$;
 F is a trained classifier with some hyperparameters;

$\mathbb{D} = (X_1, Y_1), \dots, (X_n, Y_n)$ a collection of a time series X paired with labels $Y \in \mathbb{N}$.

If $|Y| > 2$, then multi-class classification is considered. The optimization problem is defined as a search for the best pipeline consisting of a set of machine learning models, feature generators, and error correction models. This task is close to the well-known CASH problem introduced by Thornton in AutoWEKA [36]:

$$f^{min}(p^*) = \min_{P \in \mathbb{P}} f(P | T_{gen} \leq \tau_g), \quad (2)$$

where:

f - objective function that characterizes the modeling quality for the considered workflow;

f^{min} - minimal value of the loss function obtained during optimization;

$\mathbb{P}$ - a set of possible pipeline structures (search space);

T_{gen} - time spent for composite pipeline design;

τ_g - time limit.

We formulate the following hypothesis:

- **Hypothesis 1.** The process of (1) transformation from the initial time series to a particular feature space and (2) the choice of a method for extracting features from this space can be effectively automated for the problem of time series classification.
- **Hypothesis 2.** An ensembled pipeline consisting of several lightweight machine learning models obtained by the AutoML approach can be more effective in time series classification tasks for time series from various data sources than the larger SOTA models under the same resource constraints.

4. Proposed approach

Approaches to solving the problem of time series classification can be formally divided into three groups, shown in Fig. 1, each representing a sequence of identical operations. First, as a rule,

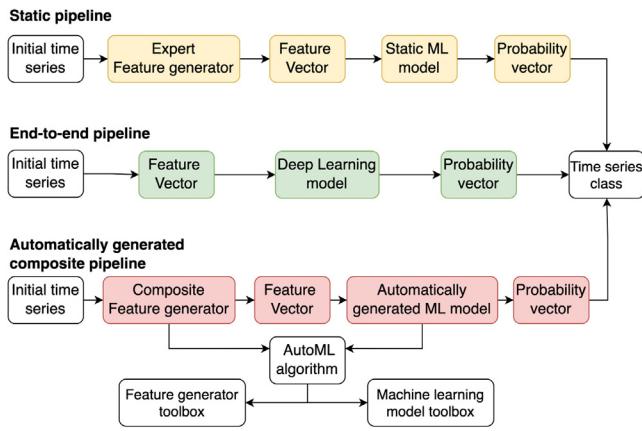


Fig. 1. Time series classification approaches.

the time series is transformed into some feature vector. This procedure can be either static or automatically generated, depending on the chosen approach. The next step is to select a classifier, which is a specific machine-learning model. The final stage is to obtain a vector (for binary classification problems) or a matrix (for multiclass classification problems) of the probabilities of class labels, which is further transformed into a vector of classes. This procedure can also be configurable and is known as probability calibration.

The automatically generated composite pipeline approach proposed in this article is based on the following concepts:

1. **The generation of the vector of features characterizing the original time series should be performed automatically and be able to add new feature spaces and combine them.** A critical aspect is the possibility of using the feature extraction model's hyperparameters, together with hyperparameters of the final pipeline of time series classification. In this article, we have implemented four methods of extracting features, and each describes a different property of time series: spectral, wavelet, topological, and quantile.

2. **The use of a pipeline consisting of feature extractors, machine learning models, and error correction models allows us to obtain a model that shows similar or higher results in comparison with SOTA approaches. This pipeline requires much less training time and has a higher degree of interpretability of the model compared, for example, with DL models.**

Using selected datasets that describe either a technological process or a natural phenomenon as experimental data and the obtained levels of automation and interpretability will make it easier for domain-specific experts to use the proposed approach without any relevant machine learning experience.

The end-to-end workflow of the FEDOT-TSC is presented in 2. There are two major blocks: (1) feature design and (2) pipeline design. The implementation details for these blocks are presented in the following subsections.

According to the scheme, the input data (Input Data block) in the form of time series goes to the Feature Design block. The selected features are the input data for the Model Design block, where the evolutionary algorithm (Evolutionary Optimizer block) selects the optimal Pipeline for solving the classification problem, and, if necessary, an error correction model is applied. The final pipelines are then applied to obtain predictions, which are interpreted with application in the corresponding block. To evaluate the quality of the intermediate results and the final predictions, the Quality Analyzer block is created, implementing various evaluation metrics.

4.1. Feature generation

As was mentioned, the proposed approach includes four feature generator types: quantile, spectral, topological, and wavelet-based. This subsection describes those generators and the need for their application.

4.1.1. Spectral feature extraction

For a one-dimensional series, the basic SSA [37,38] method converts the original series into a trajectory matrix, applies a singular value decomposition (SVD) for the resulting matrix, and groups the eigenvectors among themselves. It is often possible to distinguish some additive components of the original series, such as the trend (the smooth and slowly changing part of the series), various oscillatory and periodic components, and the noise component.

The first step of the SSA algorithm is to map the time series F to a sequence of multidimensional delay vectors. Let an integer L be the length of the window and at the same time $2 \leq L \leq N/2$. Thus a window of subsequences is formed $\{f_i, f_{i+1}, \dots, f_{i+L-1}\}$ for $i = 0, \dots, N - L$:

$$\begin{aligned} X_0 &= (f_0, f_1, f_2, \dots, f_{L-1})^T \\ X_1 &= (f_1, f_2, f_3, \dots, f_L)^T \\ X_2 &= (f_2, f_3, f_4, \dots, f_{L+1})^T \\ X_3 &= (f_3, f_4, f_5, \dots, f_{L+2})^T \\ &\vdots \\ X_{N-L} &= (f_{N-L}, f_{N-L+1}, f_{N-L+2}, \dots, f_{N-1})^T \end{aligned}$$

The above matrix shows that the elements of the antidiagonals (the diagonals going from bottom left to right) are equal. This type of matrix is known as the Hankel matrix.

The number of columns in the trajectory matrix is equal to $K = N - L + 1$. We call the columns X delay vectors L , and the rows delay vectors K . The algorithm of the proposed modification of the spectral method is presented in Fig. 3.

The authors of SSA [39] advise properly selecting the window length because its influence on the SSA results is crucial. During the experiments described in Table 2, we obtained experimental confirmation that the choice of the window width critically affects the structure of the obtained trajectory matrix and, consequently, the quality of the obtained machine learning model.

Estimating the optimal trajectory matrix window width is relevant for constructing a qualitative feature space. It can be seen from the results for *ItalyPowerDemand* or *Earthquakes* that the choice of this value can significantly affect the quality of the final prediction. In the FEDOT-TSC, we applied two heuristics:

Estimation of optimal window size. A set of eigenvectors and components is generated for each of the dataset's window size variants for each time series. Eigenvectors with explained variance values less than 1% are neglected, leaving the most influential ones. Then an empirical metric corresponding to a particular value of the window size is determined by the following formula:

$$M = \frac{D_{\text{explained}}}{n_{\text{components}}} \quad (3)$$

The highest value of this metric corresponds to the best of the considered variants of the windows for spectral analysis.

Iteration through the range of window sizes. The second option is to conduct experiments on all suggested window sizes. Results for different window sizes are presented in the Table 2:

Table 2

Metric values for different window sizes calculated using chosen data sets.

Parameter	Dataset name		
	Earthquakes	ItalyPowerDemand	Lightning7
Classification task	binary	binary	multi-class
Chosen metric	roc-auc	roc-auc	f1
Metric value for 10% window length	0.543	0.916	0.58
Metric value for 20% window length	0.544	0.976	0.521
Metric value for 30% window length	0.455	0.535	0.58

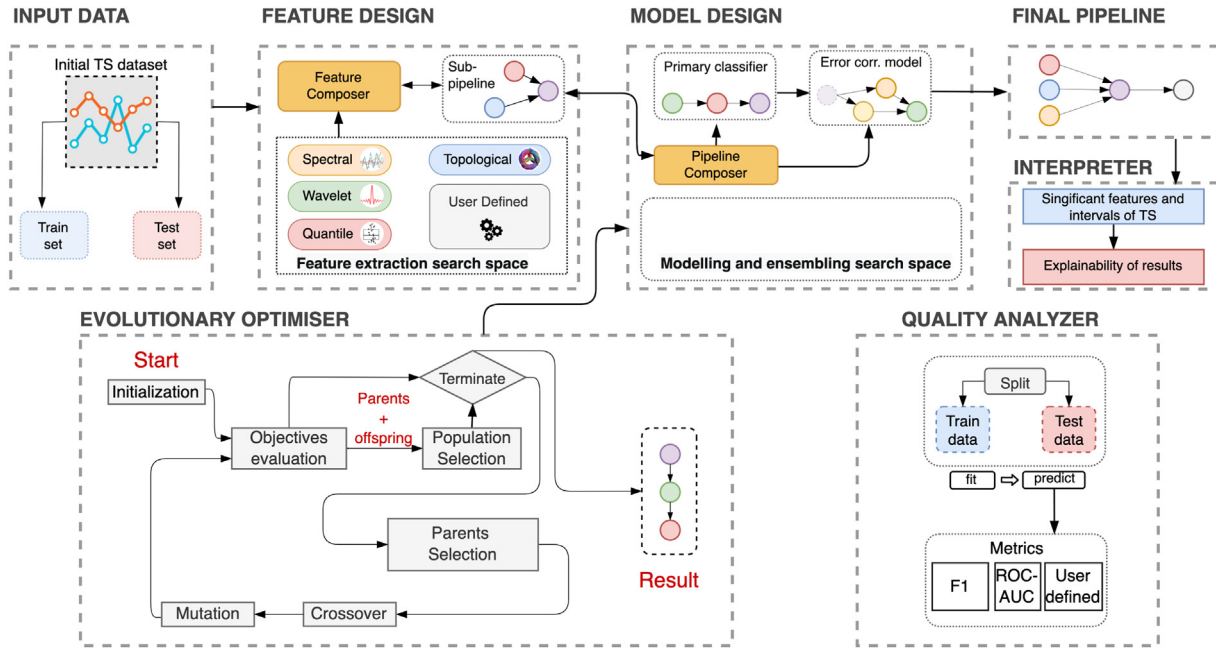


Fig. 2. The proposed concept of the end-to-end solution for the series classification problem.

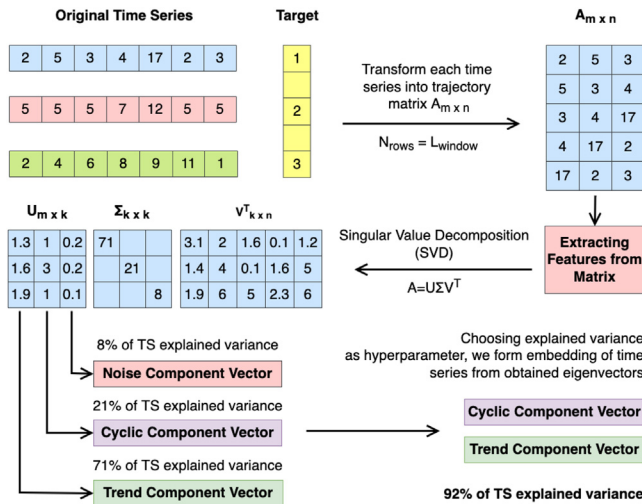


Fig. 3. Algorithm for spectral classification.

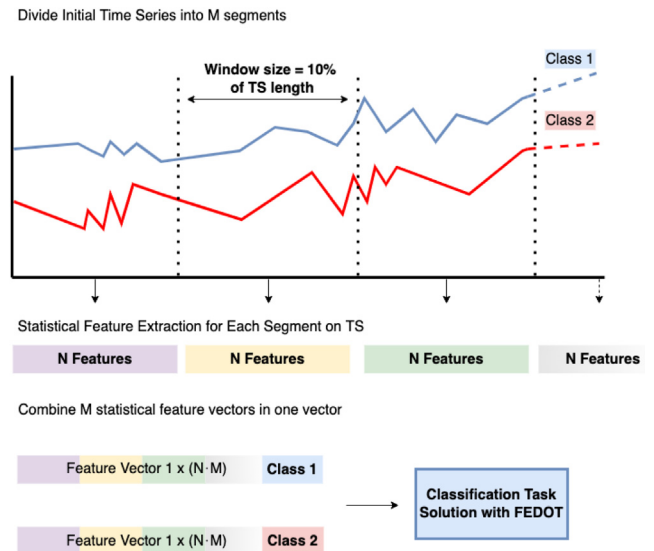


Fig. 4. Algorithm for quantile method.

4.1.2. Quantile feature extraction

This method of generating features can be used independently and as part of another method (described below in the description of the spectral feature extractor). The idea of the method (Fig. 4) is straightforward: for each time series, its feature vector is formed, which includes the values of statistical features calculated over the length of the entire series.

The statistical features given below are used to form the feature vector of each time series. If the obtained statistical feature's variance is equal to or almost zero, then such a feature is removed from the final feature vector. This method can be modified by using window methods, which may allow tracking the variability of features over time. As a feature, we use the following statistical

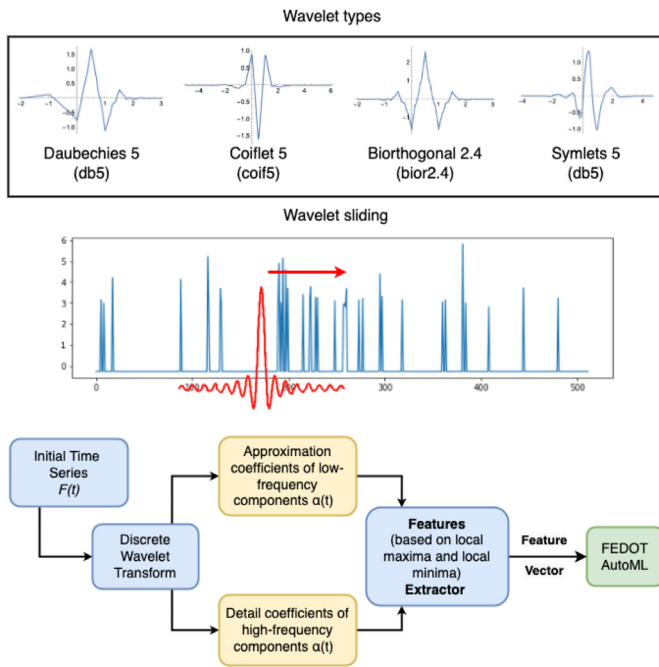


Fig. 5. Algorithm for wavelet transform.

values: mean and median values, standard deviation and dispersion values, minimum and maximum values, number of non-zero values, 5%, 25%, 75%, and 95% quantiles.

4.1.3. Wavelet transform feature extraction

This feature generation method for time series is based on the Wavelet Transform.

Wavelet transformations are time–frequency transformations using wavelets (mathematical basis functions localized in both time and frequency). They are similar to Fourier transforms, with the difference that Fourier transforms are localized only in frequency.

The approach we use to extract features from a frequency-based domain is supposed to use peaks (local maxima) or troughs (local minima) in the data obtained after the wavelet transform. As hyperparameters in this method, we use a threshold value (minimal peak value) according to which the peak (or trough) should be higher (or lower).

The final feature vector is formed using various combinations of such features (the number of peaks above a particular value, the distance between peaks with the same value, and others). Algorithms that exploit the described approach are depicted in Fig. 5.

4.1.4. Topological data analysis feature extraction

Topological Data Analysis (TDA) can broadly be described as a collection of data analysis methods that find structure in data. It includes: clustering, manifold estimation, nonlinear dimension reduction, mode estimation, ridge estimation, and persistent homology [40]. For specific time series, for example, related to EEG data analysis, the use of spectral or other features is ineffective. The reason is that specific geometric patterns in the series' structure characterize the series best. Those patterns can be described as a feature vector using the module proposed in this section.

The main idea of TDA presented in Fig. 6 is to use the topological and geometrical properties of input data, for example, its shape or connectivity, to extract informative features, which could be used further by other classification algorithms.

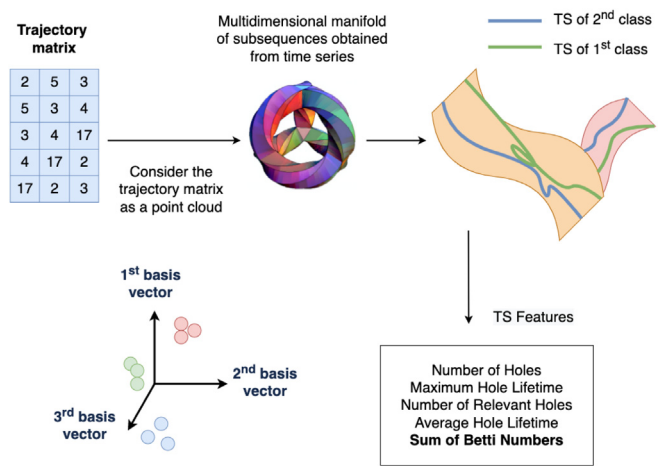


Fig. 6. Algorithm for topological data analysis and time series classification.

Persistent homology is one way to describe such features numerically. It considers data as a point cloud and tries to detect the presence of holes using discretization and triangulation of the initial data space with simplicial complexes.

Several features can be obtained in FEDOT-TSC using a TDA-based generator:

- **Number of Holes:** The first feature (or set of features) is the number of holes for each homology dimension, which can be easily calculated with PD. In the 0th dimension, the holes indicate connected components, that is, vertices along with the filtration.
- **Maximum Hole Lifetime:** The maximum hole lifetime in each dimension helps identify the most significant hole in each dimension, which is also the farthest point from the diagonal in PD. Note that the one hole that lasted the longest in the filtration indicates a prominent shape-based feature of the data set.
- **Number of Relevant Holes:** The relevant holes for each dimension are the wholes whose lifetime equals the maximum hole lifetime with some ratio. In other words, they are the points at a distance similar to that of the PD diagonal as the farthest point.
- **Average Hole Lifetime:** The average hole lifetime for each dimension shows whether the data consist of mostly short-lived holes or vice versa, the long-lived holes.
- **Sum of Betti Numbers:** The Sum of Betti Numbers can be considered an integral of the Betti Curve for each dimension. This feature is introduced to handle the structure of Betti curves.

There is a larger number of topological features that can be investigated. However, not all of them will have good interpretability concerning the original time series from which the trajectory matrix was obtained. Since FEDOT-TSC is focused on obtaining explainable results, it was decided to use a limited number of topological features.

4.2. Design of composite pipeline

4.2.1. Evolutionary optimization of classification pipeline

The task of automated machine learning for classification has many solutions in the AutoML field [41]. However, to effectively solve the time series classification task, the feature extraction stage should also be optimized together with the model. To

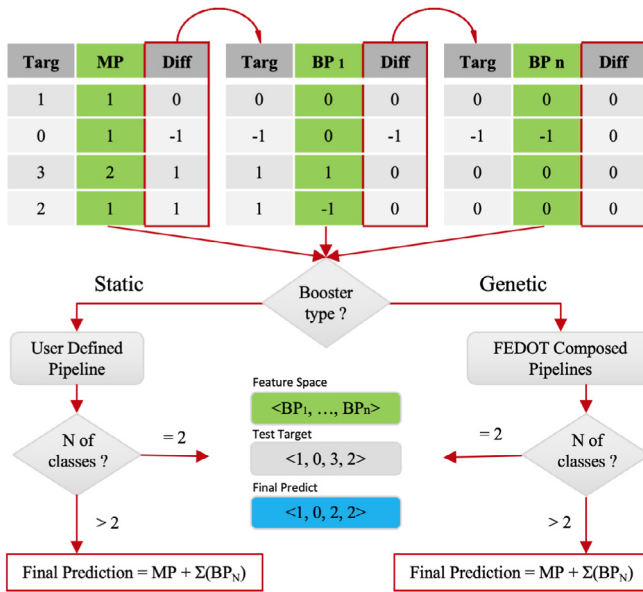


Fig. 7. Principle of error correction algorithm.

achieve this goal, we used a self-developed evolutionary optimizer [16] that automatically creates and optimizes composite machine-learning pipelines with variable-shaped structures.

It solves the generalized problem of structural learning - for a given dataset, a solution is constructed as a graph (DAG), whose nodes represent ML models, preprocessing procedures, and data transformation. The structure of this graph and the parameters of each node are trained.

Creating the most appropriate pipeline structure for a particular problem is done automatically. For this purpose, the evolutionary algorithm creates a population of many ML pipelines. It sequentially searches for the best solution, applying the evolution methods - mutation and crossover, while avoiding undesirable overcomplication of model structure (by using regularization procedures and multi-objective approaches).

The application of an abstract evolutionary optimizer to the time series classification problem requires (a) the modification of the search space (adding feature extraction algorithms and its hyperparameters); (2) implementation of wrappers for input data processing; (3) implementation of post-processing and interpretation stages; (4) development of an iterative procedure for error correction. For these reasons, we consider the obtained approach to be a separate tool.

4.2.2. Error correction model

The essence of the proposed approach is to iteratively perform compositing with the step of “freezing” the previous model. Each subsequent model is trained on the previous model error, allowing to obtain the correction of the initial prediction error. The algorithmic core is the same optimizer described in Section 1.

The error correction model is a regression model where the target for each subsequent step is the arithmetic difference between the last target and the last prediction (Fig. 7).

For multiclass classification, a multivariate regression problem is solved since the prediction is a probability matrix, and the resulting prediction is the sum of the predictions in each error correction step (Eq. (4)).

For binary classification, the results of each correction step are ensembled using another regression model. The equation for

the final prediction based on several cycles of error corrections is presented below.

$$FP = MP + \sum(BP_N) \quad (4)$$

where:

FP - final prediction;

MP - initial model prediction;

BP - error correction model prediction.

4.3. Composite model interpreter

Interpretability of the obtained results is one of the advantages of the proposed approach. The interpretation block is based on the estimation of the significance of the extracted features.

Several methods can be applied to estimate the significance of each feature. The choice of the specific one depends on the type of features and target data:

- *Kendal rank test* can be used in the case where both features and the target are not of binary type;
- *Kolmogorov-Smirnov test* can be applied for cases where features or targets are represented by binary data type;
- *Exact Fisher test* application can be used when both features and targets are binary.

The result of the Kendal rank test application to the feature space is a calculation of p-values, based on which two hypotheses of feature relevancy for predictions are being tested. Thus, some generated features are filtered out, while others are left for further analysis.

4.4. Summary of the FEDOT-TSC approach

In this section, we summarize the proposed time series classification approach.

The initial step is to read (1) the input data in the format of a time series and (2) set up the configuration file for the AutoML session.

In the next step, the data are preprocessed: this stage includes filling gaps and deleting incorrect data. Then, the splitting into train and test samples takes place.

After that, the feature extraction step starts, which results in a vector of features characterizing the original time series that is generated using all implemented approaches.

Then the composite model consisting of feature generators, machine learning, and error correction models is created using evolutionary optimization. The hyperparameters of the feature generation model are tuned together with the hyperparameters of the classification model.

The complete flowchart of the algorithms behind FEDOT-TSC is shown in Fig. 8. The following designations are used in the scheme:

- W_{long}^{tr} - width of the trajectory matrix window as a ratio to length of the time series;
- $W_{biort}^{wavelet}$ - biorthogonal type of wavelet;
- W_{long}^{mov} - the width of the sliding window, as a ratio to the length of the time series;
- e_{long} - default value for the topological generator;
- L_{gaps} and L_{min} - thresholds for the length of the time series and the number of gaps, respectively.

This approach allows for obtaining the modeling pipeline that achieves similar or better results compared to existing approaches, in much less training time, and with a higher degree of interpretability of the model (in comparison to deep learning models).

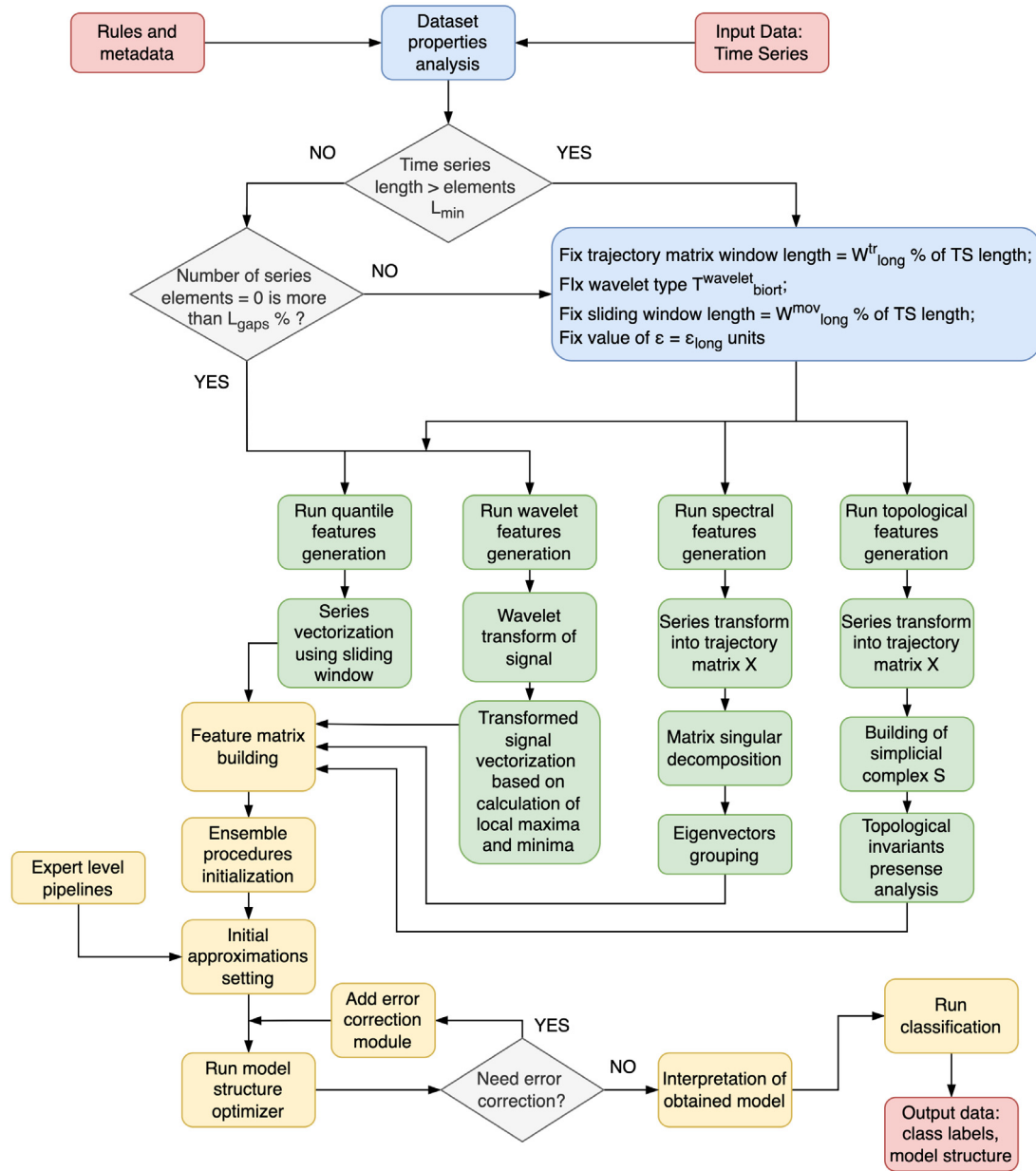


Fig. 8. Flowchart of the time series classification algorithm implemented in FEDOT-TSC.

5. Results and discussion

5.1. Setup of the experiment

The next step is to analyze the effectiveness of the proposed solution. We compared various modern time series classification algorithms (HIVE-COTE, TS-CHIEF, and InceptionTime), which can be considered the most effective today, on 65 datasets from the UEA/UCR archive [42], with the proposed approach. This experimental setup makes it possible to confirm or deny the hypotheses about the automation of feature extraction and the effectiveness of the lightweight model's ensemble.

It should be noted that the study did not compare the proposed approach with SOTA approaches in terms of algorithmic complexity for the following reasons. The results of the SOTA

approach were taken as is since it is impossible to reproduce the experiments due to the lack of data of equipments technical characteristics on which they were performed and the time of the experiments. Thus, the lower values of the quality metrics of existing SOTA approaches, which could have been obtained in our experiment, may create a false impression of the quality of the algorithms' performance. Also unclear is the question of comparing model complexity with each other since it is not obvious how to compare the number of hyperparameters of models such as InceptionNet, OS-CNN (which are deep learning models) and models such as HIVE-COTE, FEDOT-TSC (which are ensemble models) with each other.

For each dataset, 15 min are allocated for the generation of composite machine learning models and 5 min for the selection of an error correction model.

The first stage of numerical experiments is organized as follows:

- The combination of quantile feature generator and random forest model is used as a baseline;
- The best result obtained using the proposed approach is compared with the best results of other time series classification models. We use ROC-AUC as a quality metric for binary classification and F1 for multi-class classification;
- The final stage is ensembling features for models with the best values of quality metrics into a single feature matrix, for which a similar model selection procedure is performed.

The following hyperparameters were used for the proposed approach:

Hyperparameters for FEDOT-TSC: timeout: 15 min, max-depth: 10, max-arity: 4, cv-folds: 3, stopping-after-n-generation: 20.

Hyperparameters for Spectral Experiment. The spectral experiment requires the definition of the window length. It is well known that this parameter must be no more than 50% of the time series length [39]. So, for our approach for every dataset, we have used three window sizes: 10, 20 and 30% of time series length.

Hyperparameters for Topological feature generator. For this type of experiment for topological data analysis, it is necessary to define appropriate parameters, such as:

- maximum simplex dimension: [2];
- epsilon value: [100];
- persistence parameters: [None];
- window length: [same as for spectral generator].

Hyperparameters for wavelet feature generator. The experiment that involves discrete wavelet transformations requires defining the exact types of wavelet. In our case, we have chosen Daubechies, Symlets, Coiflets, and Biorthogonal wavelets.

All used feature generators and operations, as well as input and output data streams, are presented in Fig. 8.

5.2. Results of experiments

Time series classification experiments with FEDOT-TSC were conducted for 15 min for each dataset. The results were compared to state-of-the-art algorithms and combined into two tables: first for binary datasets and second for multiclass datasets.

Table 5 contains visualizations of the best pipelines obtained with FEDOT-TSC for each dataset.

Tables 3 and 4 represent the summary of the comparison of the proposed approach with state-of-the-art competitors.

According to the results, the cases can be divided into three groups:

- The proposed approach is inferior to the best available solution.
- The proposed approach is superior to the best available solution.
- The proposed approach showed a similar result with the best available solution.

For binary datasets, “single” models based on specific generators, which are close to or superior to SOTA, show the highest efficiency. It can be assumed that for “simple” datasets (with a small number of classes), the priority strategy is to search for one most suitable feature space and further configure the model’s hyperparameters.

The opposite situation is observed for multiclass datasets: ensembles of models that include different feature spaces are more effective than existing approaches. As a result, we can conclude that the more “complex” (more classes) a data set is, the higher the priority of searching for a complex composite model.

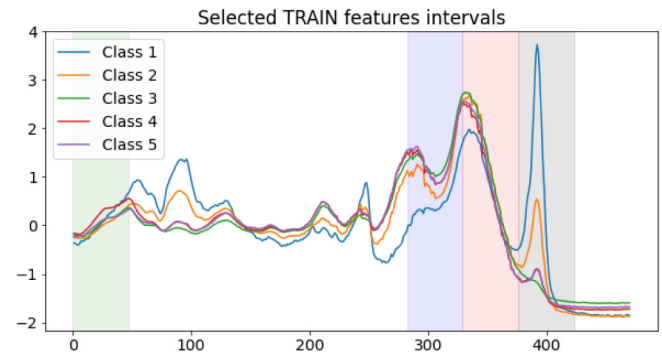


Fig. 9. Significant intervals for train subset of Beef.

Our approach outperforms SOTA results in 40 percent (26 of 65 datasets). At the same time, in 50 percent of the cases, our approach surpasses or is close (the difference is not more than five-hundredths of the metric value) to the best SOTA algorithm. Besides, looking at the average metrics values, it is possible to note that our approach loses at most 0.05 from the value of metrics of the SOTA approach. Thus, the proposed approach has shown to be effective, as it outperformed existing approaches in many experiments given the time limit and automated approach.

5.3. Analysis and interpretation

In this section, we analyze the results of the experiments. Also, we aim to interpret the pipelines (as models and feature extractors), estimate the statistical significance of obtained feature spaces, and demonstrate the relationships between features and the original time series. It is required to show to what extent we were able to confirm or refute the hypotheses that were put forward in Section 3

5.3.1. Interpretation of the features

Example of feature selection for Beef and ItalyPowerDemand datasets

To begin with, let us consider the obtained features and their relationship with the original time series. We can use datasets for which the proposed approach showed a better solution than the existing approaches. As examples, consider the Beef and ItalyPowerDemand datasets.

For the Beef dataset, the quantile extractor was used with a sliding window equal to 10% of the length of the original time series. For the ItalyPowerDemand dataset, the spectral extractor with a sliding window equal to 10% of the original time series length was chosen.

For each time series segment, the quantile extractor in window selection mode generates a set of statistical features, such as q5, q25, q75, mean, sum, min and max, var, and diff on interval. It has 15 features in total, so some have a high significance level for the classification task, while others do not.

As was mentioned and depicted in Fig. 3, the algorithm for spectral feature generation exploits SVD to obtain orthogonal components of the initial time series. Those components could be interpreted as noise, cyclic, or trend components.

Fig. 9 and represents the time intervals where the significant features are concentrated for the Beef dataset.

This way, the significant intervals for two subsets are [282 - 329] (blue interval on the train subset) and [376 - 423] (gray interval on the train subset). Those intervals represent the window size for trajectory matrix construction that was selected at the moment. It should be noted that during the experiment, the algorithm investigates several predefined window sizes. Significant features for the intervals are the following:

Table 3

Results of comparison for proposed FEDOT and SOTA solutions (binary classification, ROC-AUC metric used).

Dataset	SOTA result	SOTA algorithm	Baseline model	FEDOT result	Feature generation algorithm
BirdChicken	0.999	TS-CHIEF	0.800	1.0	Quantile
Chinatown	0.993	ROCKET	0.896	0.995	WindowQuantile
Computers	0.927	InceptionTime	0.744	0.766	Quantile
Coffee	1.0	OS-CNN	0.933	1.0	WindowSpectral
DistalPhalanxOutlineCorrect	0.914	ROCKET	0.769	0.779	WindowQuantile
Earthquakes	0.693	ProximityForest	0.509	0.740	Quantile
ECG200	0.957	InceptionTime	0.820	0.894	Ensemble: Quantile, WindowQuantile
FordA	0.994	WEASEL	0.705	<i>0.970</i>	Spectral
GunPointAgeSpan	0.999	TS-CHIEF	0.968	0.987	Statistical
GunPointMaleVersusFemale	1.0	OS-CNN	0.997	1.0	Spectral
GunPointOldVersusYoung	1.0	OS-CNN	1.0	1.0	Spectral
Ham	0.706	OS-CNN	0.600	0.724	WindowQuantile
Herring	0.686	STC	0.653	0.686	Ensemble: Topological, WindowQuantile
ItalyPowerDemand	0.992	TS-CHIEF	0.723	0.993	Ensemble: WindowSpectral, ECM
Lightning2	0.928	InceptionTime	0.629	0.794	Window Spectral
MiddlePhalanxOutlineCorrect	0.928	ROCKET	0.708	0.824	Window Quantile
MoteStrain	0.984	HIVE-COTE v1.0	0.804	0.891	Spectral
PhalangesOutlinesCorrect	0.929	InceptionTime	0.711	0.818	Window Quantile
PowerCons	1.0	TSF	0.950	1.0	Window Spectral
ProximalPhalanxOutlineCorrect	0.946	InceptionTime	0.709	0.878	Window Quantile
ShapeletSim	1.0	HIVE-COTE v1.0	0.489	1.0	Topological
SonyAIBORobotSurface1	0.998	ResNet	0.849	0.892	Window Quantile
SonyAIBORobotSurface2	0.997	ResNet	0.770	0.824	Window Quantile
Strawberry	0.997	ROCKET	0.905	0.977	Spectral
ToeSegmentation2	0.995	HIVE-COTE v1.0	0.622	0.869	Spectral
TwoLegECG	1.0	ResNet	0.846	0.919	Quantile
Wafer	1.0	TS-CHIEF	0.944	1.0	Quantile
WormsTwoClass	0.904	BOSS	0.652	0.715	Topological
Yoga	0.975	S-BOSS	0.730	0.898	WindowQuantile
Average values	0.946		0.774	0.891	

Results outperforming SOTA are shown in bold.

Results close to SOTA are shown in italics.

Baseline model is QuantileGenerator + Scaling + Random Forest.

- mean_on_interval;
- lambda_less_zero_on_interval;
- min_on_interval;
- median_on_interval;
- q5_on_interval;
- q75_on_interval;
- sum_on_interval.

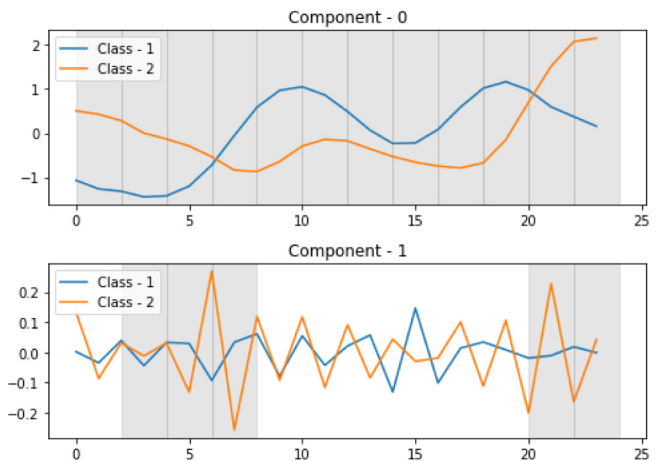
The success of the spectral-window algorithm on the ItalyPowerDemand dataset is achieved because extraction of spectral components at this particular window size allows capturing time intervals where classes are distinguishable.

There are five intersections at the beginning and end for both components. A greater level of interpretability of the obtained results can be gained from the plot where each time series component for each class is presented with significant intervals. (Fig. 10).

Analysis of the first components of each series, which can be interpreted as a global trend, shows that they are significantly distinguishable regardless of the interval. It is useful from the point of view of machine learning models' interpretability, but we can also add additional features in the second component, as shown below. The bottom chart on Fig. 10 is the representation of this idea: on the intervals [2, 8] and [20, 24] there are component oscillations as well as initial time series.

Example of feature selection for Lightning, EthanolLevel, and ElectricDevices datasets

Dataset Lightning is a relatively complex and unique case of time series, especially in terms of the proposed spectral-window approach. In this case, we observed that 11 spectral components were obtained per time series during extraction. Only three of them are presented in Fig. 11. The rest of the extracted components look similar to component 10 and represent some noise in the time series. An enormous quantity of components may also be

**Fig. 10.** Significant intervals for each component ItalyPowerDemand classes.

caused by the fact that we are not dealing with initial time series, but instead, we are using their power density. It is an example of how the model could be overcomplicated by the spectral method and subsequent feature generation. In this case, the main idea for further research is to find an additional method of reducing the number of components based on something other than the explained variance.

On the other hand, there is the case of the EthanolLevel dataset. It can be seen that the proposed approach in spectral-window mode did not perform as well as in quantile mode with the error correction model. It is visible in Fig. 12 that time series are not distinguishable either in their natural form or in spectral components. The visualization of the extracted components discloses this misconception.

Table 4
Results of comparison for proposed FEDOT and SOTA solutions (multi-class cases, F1 metric used).

Dataset	SOTA result	SOTA algorithm	Baseline model	FEDOT result	Feature generation algorithm
ACSF1	0.901	OS-CNN	0.733	<i>0.851</i>	WindowQuantile
Adiac	0.851	OS-CNN	0.011	0.776	Ensemble: Quantile, Topological
ArrowHead	0.876	cBOSS	0.635	0.803	WindowSpectral
Beef	0.768	OS-CNN	0.572	0.828	Quantile
Car	0.914	ROCKET	0.668	0.933	Ensemble: WindowSpectral, WindowQuantile, Topological
ChlorineConcentration	0.850	OS-CNN	0.522	0.863	WindowQuantile
CricketX	0.855	InceptionTime	0.469	0.707	WindowQuantile
CricketY	0.863	OS-CNN	0.452	0.71	WindowQuantile
CricketZ	0.859	OS-CNN	0.515	0.729	Ensemble: WindowSpectral, WindowQuantile, Topological
Crop	0.791	InceptionTime	0.647	0.798	Ensemble: WindowSpectral, WindowQuantile
DistalPhalanxTW	0.863	OS-CNN	0.590	0.687	WindowQuantile
DistalPhalanxOutlineAgeGroup	0.808	TS-CHIEF	0.779	<i>0.758</i>	Ensemble: Quantile, Spectral, WindowQuantile
ECG5000	0.597	TS-CHIEF	0.008	0.933	recurrence
ElectricDevices	0.868	ROCKET	0.649	0.725	Ensemble: Quantile, WindowQuantile
EOGVerticalSignal	0.811	InceptionTime	0.373	0.475	Ensemble: Quantile, WindowQuantile
EthanolLevel	0.875	InceptionTime	0.343	0.792	Ensemble: Quantile, ECM
FaceFour	0.999	TS-CHIEF	0.579	<i>0.977</i>	Ensemble: Topological, Quantile
Haptics	0.521	STC	0.350	0.490	Ensemble: Topological, Quantile
InlineSkate	0.668	WEASEL	0.341	0.525	Ensemble: Topological, WindowQuantile, Wavelet
LargeKitchenAppliances	0.954	ResNet	0.766	0.816	Ensemble: Quantile, Topological
Lightning7	0.818	InceptionTime	0.497	<i>0.779</i>	Quantile
Mallat	0.976	TS-CHIEF	0.739	<i>0.962</i>	Ensemble: Wavelet, WindowQuantile
Meat	0.994	ResNet	0.770	0.902	Ensemble: Topological, Spectral
MiddlePhalanxOutlineAgeGroup	0.653	ROCKET	0.521	<i>0.606</i>	Ensemble: Spectral, Topological, Quantile
MiddlePhalanxTW	0.410	ResNet	0.442	0.497	Ensemble: Spectral, WindowQuantile
OliveOil	0.897	TS-CHIEF	0.640	0.932	Ensemble: Topological, Spectral
Phoneme	0.238	ResNet	0.028	0.244	Topological
Plane	1.0	OS-CNN	1.0	1.0	WindowQuantile
ProximalPhalanxOutlineAgeGroup	0.840	OS-CNN	0.831	0.847	Ensemble: Wavelet, WindowQuantile
ProximalPhalanxTW	0.793	OS-CNN	0.725	0.844	Ensemble: Wavelet, WindowQuantile
RefrigerationDevices	0.790	HIVE-COTE v1.0	0.498	0.545	WindowQuantile
Rock	0.848	STC	0.493	0.880	Ensemble: Topological, WindowQuantile
ScreenType	0.755	ResNet	0.420	0.484	Ensemble: Topological, WindowQuantile, Wavelet
SwedishLeaf	0.971	OS-CNN	0.810	0.904	Ensemble: Topological, Spectral
SyntheticControl	0.999	TS-CHIEF	0.912	0.999	Ensemble: Quantile, Spectral
Trace	1.0	OS-CNN	1.0	1.0	Spectral
UMD	0.993	OS-CNN	0.892	1.0	Ensemble: WindowQuantile, WindowSpectral
Average values	0.825		0.574	0.776	

Results outperforming SOTA are shown in bold.

Results close to SOTA are shown in italics.

Baseline model is QuantileGenerator + Scaling + Random Forest.

The most significant features, in this case, are `lambda_less_zero_on_interval` and `sum_on_interval`.

It should also be mentioned that exploiting the spectral-window feature extractor can produce anomalies, as was with the *ElectricDevices* dataset (Fig. 13).

In this case, the spectral-window feature generator extracted only one component of every time series, which is quite abnormal behavior that must be investigated further. Although these time series are visually distinguishable, their spectral components are not, leading to inaccurate classification. 15 interval-based features can be considered the most important according to the significance test: min, median, mean, max, var, std, q5, q25, q75, q95, sum, dif_min, dif_mean, dif_max, `lamb_less_0_on_interv`.

The large quantity of significant features is the consequence of the straightforward difference of time series in their natural form.

5.3.2. Analysis of generated pipelines

It can be seen that most of the pipelines obtained (Table 5) contain no more than five nodes, each representing a single operation.

The critical point is that most of the experiments carried out were successful compared to SOTA solutions: the proposed framework shows superior or equal results on datasets, where deep learning approaches took the leading positions.

For such datasets as *Beef*, *Earthquakes* *ItalyPowerDemand*, *PowerCons*, *Trace*, *Wafer*, and *FordA* the proposed approach has shown that these datasets do not require complex architecture

to obtain equal or superior modeling quality results. For the other part of the datasets, sufficient classification quality could be obtained only using complex, non-linear models (*FordA*). Furthermore, the last possible scenario is when modeling quality could be significantly improved with engineering solutions, for example, an error correction model.

As we can see, the main advantage of the proposed approach is functional flexibility: due to its algorithmic evolutionary core, it can produce simple or complex pipelines (*FordA*) depending on the data structure. In most cases, there is no need for great algorithmic complexity (*PowerCons*, *Trace*, *Wafer*, *ItalyPowerDemand*).

6. Conclusions and future work

This paper proposes an approach to the problem of classifying time series, allowing getting results close to or better than SOTA in no more than 15 min for a significant part of UCR/UEA datasets.

We propose using a combination of feature extractors with an ensemble of lightweight models obtained by AutoML. The design of feature extractors integrated into a machine learning pipeline can be effectively automated for the time series classification problem. However, as shown in Table 2, the choice of the hyperparameter value for the feature extraction method can significantly impact the final value of the quality metric. Window selection and validation options were added to the proposed approach to make tuning the extractors more flexible.

Our approach has the following advantages:

Table 5
Best performed pipelines for some of selected dataset.

Dataset	Pipeline	Dataset	Pipeline
EthanolLevel		FordA	
Earthquakes		Lightning7	
ElectricDevices		PowerCons	
Beef		Trace	
ItalyPowerDemand		Wafer	

qda - a classifier with a quadratic decision boundary; **lgbm** - a gradient boosting model; **rf** - random forest classifier; **poly features** - polynomial features transform; **fast ica** - a fast algorithm for Independent Component Analysis; **logit** - Logistic Regression classifier; **pca** - principal component analysis; **simple imputat.** - simple imputation.

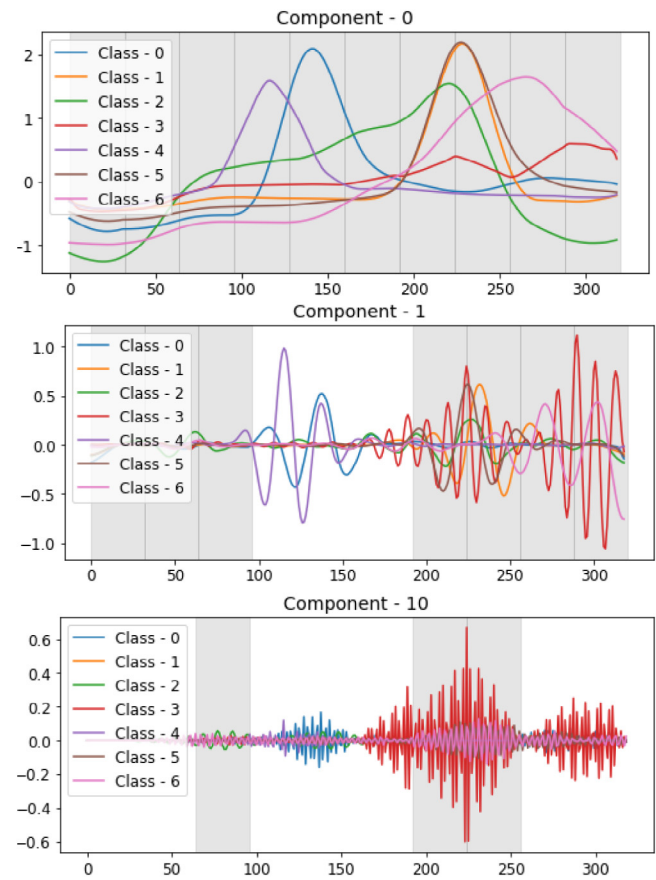


Fig. 11. Significant intervals for Lightning7 time series spectral components.

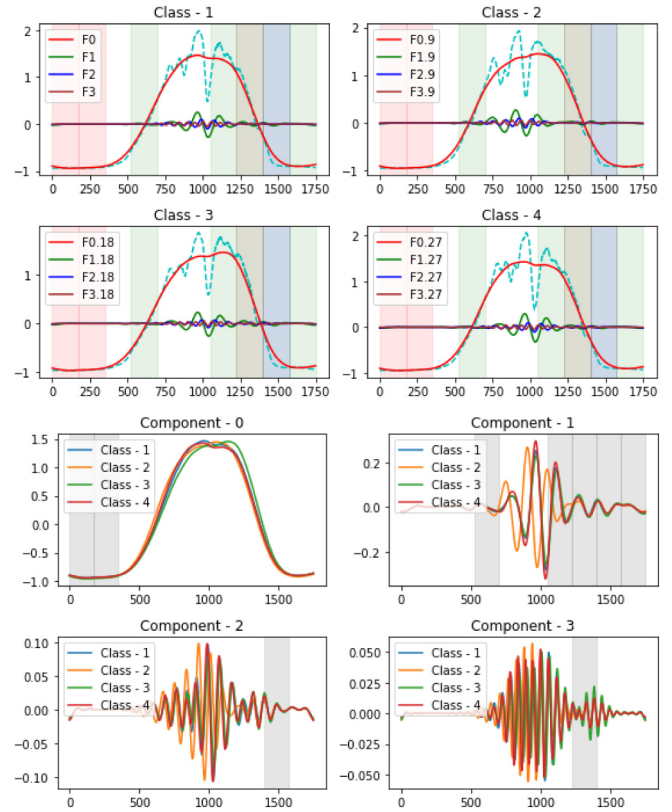


Fig. 12. Significant intervals for EthanolLevel and its components.

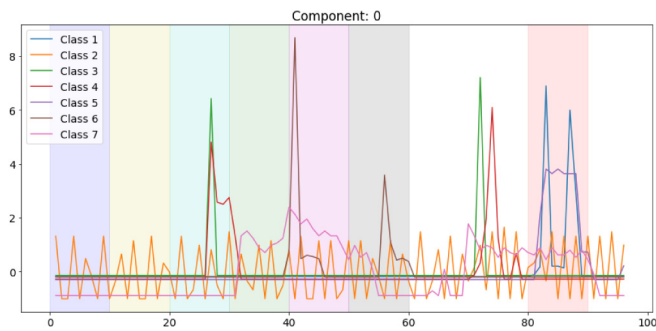


Fig. 13. Significant intervals for ElectricDevices (Component 0).

- It can be used as a tool for obtaining an initial machine-learning pipeline for time series classification.
- Pipeline structure and quality metric changes analysis is a useful tool to assess the “complexity” of the required classification model.
- The architecture of the resulting pipelines can vary from fairly simple linear models (consisting of a feature generator and a classifier model) to pipelines with a nonlinear structure. It allows for saving time and resources during the modeling without using complex models with static architecture.
- For a particular set of datasets the pipelines obtained using our solution show comparable or higher modeling quality compared to existing solutions.

Nevertheless, although it demonstrates efficiency, there are the following limitations:

- As was shown during the experiments, the set of proposed methods for feature extraction requires refinement and the inclusion of new methods.
- The automation of some of them requires refinement using expert knowledge, for example, topological feature generator requires specific knowledge from the field of topological data analysis.
- Due to the stochasticity of evolutionary optimization methods used as an algorithmic core for creating pipelines, the quality of the final pipeline and the convergence time to the optimal solution is related in a non-linear way. This fact makes it difficult to estimate the time of stopping training and the resources needed to improve the effectiveness of the proposed method.

As a result, our method has shown its efficiency on industrial data of different natures. The average gain in quality metric value between our solutions and existing solutions is about 5%. As an AutoML solution, the proposed approach can be considered a step toward filling the gap in the field of automatic classification of time series. Future development of the framework aims to improve the evolutionary algorithmic kernel and implement new feature generators and ensembling methods.

CRedit authorship contribution statement

Ilia Revin: Conceptualization, Methodology, Software, Writing – original draft. **Vadim A. Potemkin:** Visualization, Investigation, Software, Writing – original draft. **Nikita R. Balabanov:** Investigation, Validation. **Nikolay O. Nikitin:** Conceptualization, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Lya E. Revin, Vadim A. Potemkin, Nikita R. Balabanov, Nikolay O. Nikitin reports financial support was provided by Analytical Center for the Government of the Russian Federation (IGK860000000D730321P5Q0002), agreement No. 70-2021-00141.861.

Data availability

Data will be made available on request

Acknowledgments

This work was supported by the Analytical Center for the Government of the Russian Federation (IGK 000000D730321P5Q0002), agreement No. 70-2021-00141.

References

- [1] Patrick Schäfer, Ulf Leser, Fast and accurate time series classification with weasel, in: *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, 2017, pp. 637–646.
- [2] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, Pierre-Alain Muller, Deep learning for time series classification: a review, *Data Min. Knowl. Discov.* 33 (4) (2019) 917–963.
- [3] Johann Faouzi, Time series classification: A review of algorithms and implementations, *Mach. Learn. (Emerg. Trends Appl.)* (2022).
- [4] Monica Arul, Ahsan Kareem, Applications of shapelet transform to time series classification of earthquake, wind and wave data, *Eng. Struct.* 228 (2021) 111564.
- [5] Rashmi Bhardwaj, Varsha Duhoon, Study and analysis of time series of weather data of classification and clustering techniques, in: *International Conference on Innovative Computing and Communications*, Springer, 2021, pp. 257–270.
- [6] Lei Lin, Beilei Xu, Wencheng Wu, Trevor W. Richardson, Edgar A. Bernal, Medical time series classification with hierarchical attention-based temporal convolutional networks: A case study of myotonic dystrophy diagnosis, in: *CVPR Workshops*, 2019, pp. 83–86.
- [7] Ane Blázquez-García, Angel Conde, Usue Mori, Jose A. Lozano, Water leak detection using self-supervised time series classification, *Inform. Sci.* 574 (2021) 528–541.
- [8] Marília Barandas, Duarte Folgado, Leticia Fernandes, Sara Santos, Mariana Abreu, Patricia Bota, Hui Liu, Tanja Schultz, Hugo Gamboa, TSFEL: Time series feature extraction library, *SoftwareX* 11 (2020) 100456.
- [9] Maximilian Christ, Nils Braun, Julius Neuffer, Andreas W. Kempa-Liehr, Time series feature extraction on basis of scalable hypothesis tests (tsfresh—a python package), *Neurocomputing* 307 (2018) 72–77.
- [10] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, François Petitjean, Inceptiontime: Finding alexnet for time series classification, *Data Min. Knowl. Discov.* 34 (6) (2020) 1936–1962.
- [11] Zhiwen Xiao, Xin Xu, Haoxi Zhang, Edward Szczerbicki, A new multi-process collaborative architecture for time series classification, *Knowl.-Based Syst.* 220 (2021) 106934.
- [12] Jiang Wu, Zixun Zhang, Yanju Ji, Suyi Li, Lin Lin, A ResNet with GA-based structure optimization for robust time series classification, in: *2019 IEEE International Conference on Smart Manufacturing, Industrial & Logistics Engineering, SMILE*, 2019, pp. 69–74, <http://dx.doi.org/10.1109/SMILE45626.2019.8965287>.
- [13] Laurent Parmentier, Olivier Nicol, Laetitia Jourdan, Marie-Eleonore Kessaci, AutoTSC: Optimization algorithm to automatically solve the time series classification problem, in: *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence, ICTAI, IEEE*, 2021, pp. 412–419.
- [14] Hailun Xie, Li Zhang, Chee Peng Lim, Evolving CNN-LSTM models for time series prediction using enhanced grey wolf optimizer, *IEEE Access* 8 (2020) 161519–161541.
- [15] Mikhail Sarafanov, Yulia Borisova, Mikhail Maslyayev, Ilia Revin, Gleb Maximov, Nikolay O. Nikitin, Short-term river flood forecasting using composite models and automated machine learning: The case study of Lena River, *Water* (ISSN: 2073-4441) 13 (24) (2021) <http://dx.doi.org/10.3390/w13243482>, URL: <https://www.mdpi.com/2073-4441/13/24/3482>.

- [16] Nikolay O. Nikitin, Pavel Vychuzhanin, Mikhail Sarafanov, Iana S. Polonskaia, Ilya Revin, Irina V. Barabanova, Gleb Maximov, Anna V. Kalyuzhnaya, Alexander Boukhanovsky, Automated evolutionary approach for the design of composite machine learning pipelines, *Future Gener. Comput. Syst.* 127 (2022) 109–125.
- [17] Usue Mori, Alexander Mendiburu, Isabel Marta Miranda, José Antonio Lozano, Early classification of time series using multi-objective optimization techniques, *Inform. Sci.* 492 (2019) 204–218.
- [18] Usue Mori, Alexander Mendiburu, Sanjoy Dasgupta, Jose A. Lozano, Early classification of time series by simultaneously optimizing the accuracy and earliness, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (10) (2018) 4569–4578, <http://dx.doi.org/10.1109/TNNLS.2017.2764939>.
- [19] Ilya Revin, Vadim Potemkin, Nikita Balabanov, FEDOT.Industrial - Framework for automated time series classification, 2022, URL: <https://github.com/ITMO-NSS-team/Fedot.Industrial>.
- [20] Jokim Labaien, Ekhi Zugasti, Xabier De Carlos, Contrastive explanations for a deep learning model on time-series data, in: *International Conference on Big Data Analytics and Knowledge Discovery*, Springer, 2020, pp. 235–244.
- [21] Tian Wang, Zhaoying Liu, Ting Zhang, Syed Fawad Hussain, Muhammad Waqas, Yujian Li, Adaptive feature fusion for time series classification, *Knowl.-Based Syst.* 243 (2022) 108459.
- [22] Anthony Bagnall, Aaron Bostrom, James Large, Jason Lines, The great time series classification bake off: An experimental evaluation of recently proposed algorithms. Extended version, 2016.
- [23] Rohit J. Kate, Using dynamic time warping distances as features for improved time series classification, *Data Min. Knowl. Discov.* 30 (2) (2016) 283–312.
- [24] Pavel Senin, Dynamic Time Warping Algorithm Review, Vol. 855, *Information and Computer Science Department University of Hawaii at Manoa Honolulu, USA*, 2008, p. 40, 1–23.
- [25] Lexiang Ye, Eamonn Keogh, Time series shapelets: a novel technique that allows accurate, interpretable and fast classification, *Data Min. Knowl. Discov.* 22 (1) (2011) 149–182.
- [26] Jason Lines, Luke M. Davis, Jon Hills, Anthony Bagnall, A shapelet transform for time series classification, in: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2012, pp. 289–297.
- [27] Michael Flynn, James Large, Tony Bagnall, The contract random interval spectral ensemble (c-RISE): the effect of contracting a classifier on accuracy, in: *International Conference on Hybrid Artificial Intelligence Systems*, Springer, 2019, pp. 381–392.
- [28] Jason Lines, Sarah Taylor, Anthony Bagnall, Time series classification with HIVE-COTE: The hierarchical vote collective of transformation-based ensembles, *ACM Trans. Knowl. Discov. Data* 12 (5) (2018).
- [29] Ahmed Shifaz, Charlotte Pelletier, François Petitjean, Geoffrey I. Webb, TS-CHIEF: a scalable and accurate forest algorithm for time series classification, *Data Min. Knowl. Discov.* 34 (3) (2020) 742–775.
- [30] Benjamin Lucas, Ahmed Shifaz, Charlotte Pelletier, Lachlan O'Neill, Nayyar Zaidi, Bart Goethals, François Petitjean, Geoffrey I. Webb, Proximity forest: an effective and scalable distance-based classifier for time series, *Data Min. Knowl. Discov.* 33 (3) (2019) 607–635.
- [31] Wensi Tang, Guodong Long, Lu Liu, Tianyi Zhou, Michael Blumenstein, Jing Jiang, Omni-scale CNNs: a simple and effective kernel size configuration for time series classification, in: *International Conference on Learning Representations*, 2021.
- [32] D. Van Kuppevelt, C. Meijer, F. Huber, A. van der Ploeg, S. Georgievskaya, Vincent T. van Hees, Mcfly: Automated deep learning on time series, *SoftwareX* 12 (2020) 100548.
- [33] Johann Faouzi, Hicham Janati, Pyts: A python package for time series classification, *J. Mach. Learn. Res.* 21 (46) (2020) 1–6, URL: <http://jmlr.org/papers/v21/19-763.html>.
- [34] Markus Löning, Anthony Bagnall, Sajaysurya Ganesh, Viktor Kazakov, Jason Lines, Franz J. Király, Sktime: A unified interface for machine learning with time series, 2019, arXiv preprint [arXiv:1909.07872](https://arxiv.org/abs/1909.07872).
- [35] Romain Tavenard, Johann Faouzi, Gilles Vandewiele, Felix Divo, Guillaume Androz, Chester Holtz, Marie Payne, Roman Yurchak, Marc Rußwurm, Kushal Kolar, Eli Woods, Tslearn, a machine learning toolkit for time series data, *J. Mach. Learn. Res.* 21 (118) (2020) 1–6, URL: <http://jmlr.org/papers/v21/20-091.html>.
- [36] Chris Thornton, Frank Hutter, Holger H. Hoos, Kevin Leyton-Brown, Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms, in: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2013, pp. 847–855.
- [37] Nina Golyandina, Anton Korobeynikov, Anatoly Zhigljavsky, *Singular Spectrum Analysis with R*, Springer, 2018.
- [38] Hossein Hassani, Allan Webster, Emmanuel Sirimal Silva, Saeed Heravi, Forecasting US tourist arrivals using optimal singular spectrum analysis, *Tour. Manag.* 46 (2015) 322–335.
- [39] Nina Golyandina, Vladimir Nekrutkin, Anatoly A. Zhigljavsky, *Analysis of Time Series Structure: SSA and Related Techniques*, CRC Press, 2001.
- [40] Larry Wasserman, *Topological data analysis*, 2016, arXiv preprint [arXiv:1609.08227](https://arxiv.org/abs/1609.08227).
- [41] Xin He, Kaiyong Zhao, Xiaowen Chu, AutoML: A survey of the state-of-the-art, *Knowl.-Based Syst.* 212 (2021) 106622.
- [42] Anthony Bagnall, Eamonn Keogh, Jason Lines, Aaron Bostrom, James Large, Matthew Middlehurst, UEA & UCR time series classification repository, 2022, URL: <http://www.timeseriesclassification.com>.